Understanding the Enveda Competition Objective

https://gemini.google.com/app/fc133513e49f5327

User prompt: i want to understand about this competition what is the objective here in simple language

Response: The goal of the Enveda CASMI 2026 competition is to predict the **2D chemical structure** of a molecule from its **tandem mass spectrum (LC-MS/MS)**.

In simple terms:

- **The Input (The Fingerprint):** When a biological sample is run through a mass spectrometer, the molecules get smashed into smaller fragments. The instrument records how heavy those fragments are and how much of each fragment exists, creating an "intensity barcode" or fragmentation pattern.
- **The Output (The Molecule):** Your machine learning model must look at that fragmentation pattern and reconstruct the original molecule as a **SMILES string** (a standard text representation of a chemical formula).
- **The Challenge:** While matching spectra against known databases works for already-discovered compounds, it fails on entirely novel or rare natural molecules. The goal is to build models that can generalize and deduce structures even for unseen molecules.

Key Mechanics

- **Ranked Guesses:** For every test spectrum, you provide up to **25 SMILES guesses** ordered from most likely to least likely, separated by semicolons.
- **Scoring (MRR@25):** Evaluated via Mean Reciprocal Rank. A correct answer at #1 gives 1.0 point, at #2 gives 0.5, down to 0.04 at #25 (and 0 if none of your 25 guesses match).
- **Evaluation Match Rule:** The metric compares only the 2D atom connectivity (using the first 14 characters of the InChIKey via RDKit tautomer canonicalization). You do not need to predict stereocenters (chirality) or specific tautomer forms correctly to receive full credit.
- **Submission Format:** A Kaggle code competition submission (notebook-only, offline, under 9 hours run-time) generating a `submission.csv` mapping `molecule_id` to your top 25 `smiles`.

User prompt: now i want to understand for winning this competition what are the knowledgs i need analyse and explain with step by step reasoning

Response: To win this competition, you must master the intersection of **Analytical Mass Spectrometry**, **Cheminformatics**, and **Sequence/Graph Deep Learning**. The task is framed as either a candidate-retrieval ranking problem or an end-to-end de novo generation problem under a Mean Reciprocal Rank (MRR@25) evaluation metric.

The foundational areas of knowledge required, along with the step-by-step reasoning for why they matter and how to execute them, are detailed below.

Step 1: Mass Spectrometry & Analytical Chemistry Fundamentals

- **Understanding the Physical Process:**
  - **Reasoning:** Tandem mass spectrometry (MS/MS or MS^2) does not measure atoms directly; it ionizes molecules into precursor ions and fractures them using collision-induced dissociation (CID). The machine outputs pairs of mass-to-charge ratios (m/z) and signal intensities.
  - **Adducts and Ionization Modes:** You must know how ions form (e.g., positive mode $[M+H]^+$, $[M+Na]^+$; negative mode $[M-H]^-$). A shift in precursor mass tells you the exact ionization state.
  - **Neutral Losses & Fragment Logic:** The numerical difference between the precursor ion and fragment peaks corresponds to neutral losses (e.g., -18.01 Da indicates the loss of $\mathrm{H_2O}$, -43.99 Da corresponds to $\mathrm{CO_2}$). These differences directly reveal chemical functional groups.
  - **Actionable Knowledge:** Implement automated feature extraction that calculates both peak m/z and neutral loss vectors ($\Delta m/z = m/z_{\text{precursor}} - m/z_{\text{fragment}}$).

Step 2: Cheminformatics & Representation Learning

- **Molecular Formulations & Canonicalization:**
  - **Reasoning:** The evaluation checks 2D planar connectivity using the first 14 characters of the InChIKey (tautomer-canonicalized via RDKit). Generating an identical molecule in a non-standard tautomer or with different stereocenters could cause an evaluation mismatch if not canonicalized correctly before submission.
  - **Molecular Fingerprints:** MACCS keys, Morgan/ECFP4/ECFP6 fingerprints, and MAP4 encode presence/absence of sub-structural fragments. High-performing MS pipelines (like CSI:FingerID, SIRIUS, ChemEmbed, and MIST) treat fingerprint prediction as an intermediate surrogate task.
  - **Actionable Knowledge:**

- Master **RDKit** routines: sanitization, SMILES canonicalization, tautomer hashing, and formula calculation.
- Filter candidate search spaces strictly by precursor monoisotopic mass and formula constraints (within 5–10 ppm tolerance).

Step 3: Deep Learning Architectures for Mass Spectra

Top-performing pipelines in CASMI benchmarks follow two main paradigms:

- **Paradigm A: Two-Stage Retrieval (Predict Fingerprints $\rightarrow$ Rank Database Candidates)**
  - **Step 1:** Train a 1D-CNN or Transformer (similar to MIST or ChemEmbed) that takes continuous $(m/z, \text{intensity})$ and neutral-loss spectra to predict a multi-label binary fingerprint vector (e.g., 2,048 to 4,096 bits).
  - **Step 2:** Query candidate molecules from large chemical libraries (PubChem, ChEMBL, COCONUT, ZINC) matching the precursor mass window, calculate their Tanimoto similarity against the predicted fingerprint vector, and rank the top 25.
- **Paradigm B: De Novo Sequence-to-Sequence (Spec2Mol / MSNovelist / MolForge)**
  - Use a transformer encoder over the spectrum tokens and an autoregressive decoder (or pretrained chemical LLM) to generate valid canonical SMILES directly.
  - **Beam Search & Chemical Validity:** Use beam search with a beam width of at least 25 to collect the top candidates, checking RDKit validity on the fly.
- **Actionable Knowledge:** PyTorch-based sequence modeling, contrastive representation learning (e.g., InfoNCE loss between spectral embeddings and molecular graph embeddings), and beam search decoding algorithms.

Step 4: Data Engineering & External Reference Databases

- **The Data Bottleneck:**
  - **Reasoning:** The provided training dataset is typically limited compared to the vastness of chemical space. Winning solutions often leverage public spectral datasets and natural product structures.
  - **External Datasets:** GNPS (Global Natural Product Social Molecular Networking), MassBank, MoNA (MassBank of North America), NIST (if accessible), MassSpecGym, and the COCONUT natural product database.
  - **Data Augmentation:** Spectra vary by collision energy and instrument noise. Augmentations include random peak intensity jittering, peak dropout (masking low-abundance fragments), and synthetic precursor shifts.

Step 5: Kaggle Competition Engineering & Rule Constraints

- **Metric Optimization (MRR@25):**
  - $$\text{MRR@25} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$
  - Getting the correct molecule at Rank 1 scores 1.0, whereas Rank 2 drops immediately to 0.5. Your rank-ordering policy must be aggressive: reserve top slots for high-confidence mass-matched database hits, followed by de novo generative candidates to cover unexplored chemical space.
- **Offline Notebook Limits:**
  - Inference must execute strictly offline without internet access within the runtime limit (typically 9 hours on Colab/Kaggle GPUs).
  - Pre-compute candidate databases, pre-index molecular weights for fast $O(1)$ precursor mass filtering, and store models as self-contained Kaggle Datasets.

Summary Checklist to Get Started

1. **Build a Local Validation Split:** Use scaffold splitting or cluster-based splitting (e.g., Butina/Murcko) so validation spectra do not share molecular scaffolds with the training set (simulating real discovery).
2. **Replicate a Baseline:** Implement a two-stage baseline (Spectrum $\rightarrow$ 2048-bit Morgan Fingerprint predictor using a 1D ConvNet/Transformer $\rightarrow$ Cosine/Tanimoto candidate ranking against a local candidate pool).
3. **Canonicalize Everything:** Ensure all training and predicted SMILES pass through the official RDKit tautomer hash function to guarantee zero formatting penalty on the leaderboard.

User prompt: A prediction is correct when it describes the same atom connectivity as the answer. Both your SMILES and the answer are passed through RDKit's tautomer canonicalization (pinned at 2026.03.3) and reduced to the first block of their InChIKey (the InChIKey14), then compared. The prediction is correct when the two first-block keys match. This means you are not penalized for getting stereocenters or tautomer forms wrong.

For example, both of the following score identically against an answer of glucose, because they reduce to the same InChIKey14 (WQZGKKKJIJFFOK): explain this simply

Response: This rule means the competition evaluates your prediction purely as a **2D skeleton diagram**, ignoring **3D arrangement (stereochemistry)** and **dynamic hydrogen shifts (tautomers)**.

1. The Three Parts of an InChIKey

An InChIKey is a 27-character digital fingerprint of a molecule, split into three blocks separated by hyphens (e.g., `WQZGKKKJIJFFOK-UHFFFAOYSA-N`):

- **Block 1 (First 14 letters):** Encodes the **atom connectivity** (which atom connects to which atom, forming the core 2D molecular backbone).
- **Block 2 (Next 9–10 letters):** Encodes **stereochemistry** (wedges/dashes, 3D orientation, cis/trans, chiral centers) and isotopic variants.
- **Block 3 (Final characters):** Encodes protonation and charge states.

The metric discards everything after the first hyphen. As long as your first 14 characters (`WQZGKKKJIJFFOK`) match the answer's first 14 characters, you receive **100% full credit**.

2. What You Can "Get Wrong" Without Losing Points

- **Stereochemistry (Chirality / 3D shape):**
  - In chemistry, D-glucose and L-galactose have the same atom connectivity, but different 3D orientations (stereocenters).
  - Normally, in strict SMILES matching, getting a wedge or dash wrong fails the match.
  - Here, stereocenters are ignored. Whether you predict D-glucose, L-glucose, or flat unassigned glucose without stereochemistry `@` symbols, their first 14 InChIKey letters are identical.
- **Tautomers (Keto-Enol shifts):**
  - Some molecules naturally exist in dynamic equilibrium by shifting a proton and a double bond back and forth (e.g., ketone vs. enol form).
  - Because RDKit runs **tautomer canonicalization** first, both forms resolve to the exact same standardized canonical backbone before generating the key.

3. What It Means for Your Modeling Strategy

- **You don't need to predict `@` or `@@` in SMILES:** You can strip or ignore chiral tags in your output SMILES during training and generation; they will not impact your score.
- **Mass spectrometry match:** Standard tandem mass spectrometry (MS/MS) measures fragment masses and neutral losses, which tell you which functional groups and bonds break—not how they point in 3D space. The metric is deliberately designed to match what mass spectrometers actually measure.

User prompt: Test set ~1,500 spectra of ~400 molecules, 1–16 spectra per molecule (median 3), all acquired on a Bruker timsTOF. Monoisotopic masses range from 157 to 1,159 Da (median 348). Every scored molecule falls into one of three novelty classes, in order of increasing difficulty: what is Bruker timsTOF explain simply and what does this term Monoisotopic masses means

Response: 1. What is a Bruker timsTOF?

A **Bruker timsTOF** is a state-of-the-art, high-resolution analytical instrument manufactured by Bruker. The name breaks down into two core technologies:

- **TOF (Time-of-Flight):** The mass analyzer. It accelerates ionized molecules down an electric flight tube toward a detector. Lighter fragments travel faster and hit the detector first; heavier fragments take longer. It delivers high mass accuracy (often down to 3–4 decimal places).
- **tims (Trapped Ion Mobility Spectrometry):** An extra separation step that happens before mass measurement. Ions are pushed against a stream of gas to separate them by their physical size and 3D cross-sectional shape (collisional cross section, or CCS), not just their mass.

Why this detail matters for the competition:

- **High Precision:** A timsTOF yields accurate mass peaks (e.g., 348.1524 instead of just 348). You can filter candidate molecules or chemical formulas within a strict error window (typically 5–10 ppm).
- **Consistent Instrument Profile:** Because all test spectra come from the same instrument family, peak shapes, fragmentation energy patterns, and detector response curves will behave consistently across the evaluation set.

2. What is Monoisotopic Mass?

The **monoisotopic mass** of a molecule is its exact molecular weight calculated using the **most abundant natural isotope** for every single atom in its formula.

Most elements on Earth exist as a mix of isotopes:

- Carbon is mostly Carbon-12 ($^{12}\mathrm{C} = 12.000000\ \mathrm{Da}$), but ~1.1% is Carbon-13 ($^{13}\mathrm{C} \approx 13.003355\ \mathrm{Da}$).
- Hydrogen is mostly $^{1}\mathrm{H}$ ($1.007825\ \mathrm{Da}$), but a tiny fraction is Deuterium ($^{2}\mathrm{H}$).

Monoisotopic Mass vs. Average Molecular Weight

Concept	How It's Calculated	Example (Ethanol, $\mathrm{C_2H_6O}$)	When It's Used
Average Mass (Periodic Table mass)	Weighted average of all natural isotopes (using 12.011 for C, 1.008 for H)	$\approx \mathbf{46.069}\ \mathrm{Da}$	Wet chemistry lab work (weighing powder on a balance)
Monoisotopic Mass	Sum of strictly the dominant light isotopes ($^{12}\mathrm{C}$, $^{1}\mathrm{H}$, $^{16}\mathrm{O}$)	$\approx \mathbf{46.04186}\ \mathrm{Da}$	High-resolution mass spectrometers (like timsTOF)

High-resolution mass spectrometers do not measure the blended average mass—they resolve individual molecules into separate isotope peaks. The **tallest, lowest-mass peak** in the parent cluster represents the molecule built exclusively out of $^{12}\mathrm{C}$, $^{1}\mathrm{H}$, $^{14}\mathrm{N}$, and $^{16}\mathrm{O}$, which is its monoisotopic mass.

3. What the Test Set Numbers Tell You

- **Range (157 to 1,159 Da, median 348 Da):**
  - The vast majority of target compounds are typical "small molecules" and secondary metabolites (drugs, lipids, natural products).
  - A median of **348 Da** means standard chemical graph/SMILES decoders and candidate retrieval databases (like COCONUT, PubChem, and ChEMBL) will cover the typical mass distribution without needing macromolecular/peptide pipelines.

User prompt: Columns Peak lists are stored as two aligned arrays: the i-th m/z pairs with the i-th intensity, with intensities normalised per spectrum so the largest peak equals 1.0. train.parquet and test.parquet use identical names for the shared columns, so preprocessing code ports directly. explain this simply with numeric examples

Response: A mass spectrum consists of two parallel lists of numbers that share the exact same length:

- `m/z` **array:** The mass-to-charge ratios (the horizontal axis of the spectrum, indicating fragment weights).
- `intensity` **array:** How abundant each fragment is (the vertical height of each peak).

What "Aligned Arrays" Means

"Aligned" means the items at index i belong together:

$$\mathrm{Peak}_i = (\mathrm{m/z}[i],\ \mathrm{intensity}[i])$$

If you inspect a single row of data, it looks like this:

```
mz        = [ 91.054,  105.070,  133.065,  151.075 ]
intensity = [  0.150,    1.000,    0.420,    0.080 ]
```

- **Peak 0:** Fragment with mass 91.054 has a relative intensity of 0.150.
- **Peak 1:** Fragment with mass 105.070 has a relative intensity of 1.000 (the base peak, tallest fragment).
- **Peak 2:** Fragment with mass 133.065 has a relative intensity of 0.420.
- **Peak 3:** Fragment with mass 151.075 has a relative intensity of 0.080.

What "Normalised Per Spectrum" Means

Raw mass spectrometers measure ion counts as large, arbitrary numbers depending on detector gain and sample concentration.

To make all spectra comparable, every raw peak intensity in a spectrum is divided by the maximum peak intensity in that specific spectrum:

$$\mathrm{intensity}_{\mathrm{normalised}} = \frac{\mathrm{raw_intensity}}{\max(\mathrm{raw_intensity})}$$

Concrete Example

Suppose the instrument detected these raw ion counts for a molecule:

Peak Index (i)	Fragment m/z	Raw Intensity (Detector Counts)	Normalized Calculation	Stored Value in Dataset
0	91.054	$75,000$	$75,000 / 500,000$	0.15
1	105.070	$500,000$ (Max)	$500,000 / 500,000$	1.00
2	133.065	$210,000$	$210,000 / 500,000$	0.42
3	151.075	$40,000$	$40,000 / 500,000$	0.08

Every spectrum is scaled between 0.0 and 1.0, with exactly one peak (the tallest) anchored at 1.0.

Why the Shared Schema Matters for Code

Because `train.parquet` and `test.parquet` share the exact same column names and array structures, you can write your feature extraction function once:

```
# Turns the aligned 1D arrays into a 2D matrix of shape (N_peaks, 2)
def parse_spectrum(row):
    mz = row['mz']
    intensity = row['intensity']
    return np.column_stack([mz, intensity])
```

The identical function works directly on both training samples and test samples without needing column mapping or extra normalization steps.

User prompt: Collision energy Collision energy governs how completely a molecule fragments: higher energy means more fragmentation and smaller fragments on average. Instruments report it differently — absolute electron volts (eV, beam-type instruments), normalised collision energy (NCE, a Thermo percentage scale), occasionally raw voltages, and often nothing at all. eV and NCE are different quantities on different scales, so the training set carries three columns: collision_energy_orig — the value exactly as the source recorded it, lossless, as a string: 40, [20 40 60], 35HCD, 6V. A multi-value entry means several single-energy acquisitions were merged into one spectrum. collision_energy_orig_units — eV, NCE, V, or unknown. Assigned from the declared string, then from documented library conventions (MSnLib is NCE, confirmed by its authors; Enveda-180 is eV), then from the instrument's analyser family. collision_energy_ev — our best-effort conversion to eV, matching the test file's convention. NCE is converted with the nominal Thermo formula (NCE × precursor_mz / 500 × charge factor), which is approximate because the true conversion depends on instrument tuning. Null where the unit is unresolved. If your model consumes collision energy, use collision_energy_ev — it is the column designed to line up with the test set. now explain this part of collision energy step by step simply

Response: Collision energy determines how hard a molecule is smashed when it enters the fragmentation chamber.

Here is the step-by-step breakdown of what it means physically, why the data is messy, and how to use it in your code.

Step 1: The Physics of Collision Energy

Inside a tandem mass spectrometer (MS/MS), precursor ions collide with an inert gas (like Nitrogen or Argon) to break apart into pieces:

- **Low Energy (e.g., 10–20 eV):** Gentle impact. Most of the original molecule stays intact. You see large fragments and weak bond cleavages.
- **High Energy (e.g., 40–80+ eV):** Violent impact. The molecule shatters completely into small, simple fragments (like tiny hydrocarbon chains or single rings).

Because different collision energies produce completely different spectra for the exact same molecule, a machine learning model benefits from knowing how hard the molecule was hit.

Step 2: The Data Problem (Mismatched Scales)

Different instrument manufacturers record collision energy using completely different units:

1. **Absolute Electron Volts (eV):** Used by beam-type instruments (like Waters, Agilent, and Bruker timsTOF). It directly measures the kinetic energy applied to the ion.
2. **Normalized Collision Energy (NCE):** A percentage-based scale popular on Thermo instruments. It scales the collision energy relative to the precursor mass so that heavier molecules receive higher energy automatically.
3. **Raw Volts (V) / Stepped Acquisitions:** Some runs use raw voltage or combine multiple runs into one spectrum (e.g., merging spectra taken at 20, 40, and 60 eV).
4. **Missing Data:** Many public reference libraries never recorded the collision energy at all.

Because $35\ \mathrm{NCE}$ is **not** equal to $35\ \mathrm{eV}$, feeding raw numbers directly into a model creates conflicting noise.

Step 3: The Three Dataset Columns Explained

To handle this mess cleanly, the organizers split collision energy into three specific columns:

Column Name	What It Contains	Example Values	What It's For
`collision_energy_orig`	The exact raw string recorded by the lab/machine.	`"40"`, `"[20 40 60]"`, `"35HCD"`, `"6V"`	Archival/lossless storage; inspection if you want custom parsing.
`collision_energy_orig_units`	The identified unit of measurement.	`"eV"`, `"NCE"`, `"V"`, `"unknown"`	Tells you what unit the original text was using.
`collision_energy_ev`	Standardized float converted to electron volts (eV).	`40.0`, `27.3`, `NaN`	**The exact column you should feed into your ML models.**

Step 4: How the Conversion to `collision_energy_ev` Works

For instruments that reported NCE (Thermo scale), the organizers approximated the absolute energy in eV using the standard formula:

$$\text{collision_energy_ev} \approx \text{NCE} \times \frac{\text{precursor_mz}}{500} \times \text{charge_factor}$$

- **Example:** If $\mathrm{NCE} = 35$, precursor $m/z = 500$, and charge $= 1$:

$$\text{Energy} \approx 35 \times \frac{500}{500} \times 1 = 35 \text{ eV}$$

- If the original unit could not be determined reliably, `collision_energy_ev` is left as **`NaN` (null)**.

Step 5: Best Practices for Your Model Pipeline

- **Always use `collision_energy_ev`:** The test set spectra were collected on a Bruker timsTOF, which natively measures collision energy in eV. Using `collision_energy_ev` aligns your training distribution with the test set.
- **Handle `NaN` values:** Because some training rows have missing units, impute missing values (e.g., using the median eV across the training set, around 20–35 eV) or pass a binary indicator mask (`is_energy_missing`) to your neural network.
- **Conditioning:** You can project `collision_energy_ev` through a small MLP or sinusoidal embedding layer and add/concatenate it to your spectrum embeddings.

User prompt: MIST-CF and SIRIUS — learned and rule-based molecular-formula annotation from MS/MS. Predicting the molecular formula first and conditioning on it is a common method for narrowing down the search space of possible structures. matchms — an open library for working with mass spectra: cleaning, curation, and similarity. GNPS propagated ("suspect") annotations — structures transferred to unidentified spectra via spectral-network similarity. Excluded from train.parquet because the labels are inferred rather than measured. MassIVE — the community repository of raw metabolomics runs: billions of unannotated MS/MS spectra. No structure labels, but a natural corpus for self-supervised pretraining. GeMS and DreaMS — the DreaMS project released GeMS, a large curated corpus of unannotated spectra mined from MassIVE, plus pretrained spectrum-embedding models. A shortcut to repository-scale pretraining. FragHub — an open data repository that merges and harmonises the major open MS/MS libraries. Overlaps heavily with train.parquet but may contain some additional spectra either experimental or predicted. i want to understand about each of this resources simply

Response: These resources make up the core ecosystem of modern computational metabolomics and mass spectrometry deep learning.

1. MIST-CF and SIRIUS

- **What they do:** Predict the exact **molecular formula** (the element counts, e.g., $\mathrm{C_{16}H_{19}N_3O_4S}$) from the spectrum before guessing the 2D bond arrangement.
- **Why it matters:** Searching billions of potential molecules by mass alone (e.g., 348.15 Da) yields thousands of candidate structures. If your model first pinpoints the exact element formula, the search space drops by ~95–99%, leaving only valid structural isomers to rank.
- **How they differ:**
  - **SIRIUS:** Uses combinatorial algorithms and physics-informed rules (fragmentation trees) to calculate which atom combinations explain the fragment peaks.
  - **MIST-CF:** A deep learning transformer that directly scores and ranks candidate formulas from spectra without needing slow fragmentation trees.

2. matchms

- **What it is:** The standard Python data processing toolkit for mass spectra (essentially "Pandas/Scikit-learn for mass spec").
- **Why it matters:** Raw spectra contain noise: stray low-intensity peaks, uncalibrated m/z drifts, duplicate peaks, and solvent artifacts. `matchms` provides modular filters to clean, normalize, peak-filter, and calculate classical spectral similarity (like cosine or modified cosine scores) in just a

few lines of code.

3. GNPS Propagated ("Suspect") Annotations

- **What they are:** "Inferred" labels assigned by an algorithm rather than confirmed by human chemists with pure standards.
- **How it works:** If an unknown spectrum is 95% similar to a known spectrum of Compound X, the GNPS platform tags the unknown as "probably a derivative of Compound X."
- **Why they are excluded from the training set:** Because they are algorithmic guesses, they contain label noise and error propagation. The competition organizers omitted them to prevent models from learning false associations.

4. MassIVE

- **What it is:** A massive global public data repository where laboratories deposit terabytes of raw LC-MS/MS data from untargeted metabolomics experiments.
- **Why it matters:** Most spectra in MassIVE have no labels (no one knows what molecule produced them). While unusable for direct supervised training, having hundreds of millions of unlabeled spectra makes it the ideal pretraining ground for self-supervised learning (masking peaks and predicting them back, similar to BERT pretraining).

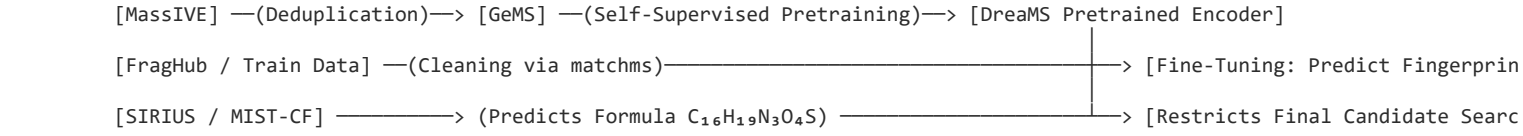
5. GeMS and DreaMS

- **What they are:** A curated foundation model and dataset shortcut built on top of MassIVE.
  - **GeMS (Genomic & Metabolomic Spectra):** A deduplicated, quality-filtered subset of tens of millions of unannotated spectra extracted from MassIVE.
  - **DreaMS:** A transformer foundation model pretrained on GeMS using self-supervised objectives (predicting masked peaks and chromatographic retention).
- **Why it matters for you:** Training a transformer on millions of spectra from scratch takes weeks of compute. DreaMS releases pretrained weights that map any mass spectrum into an information-rich embedding vector, giving you an immediate feature extractor.

6. FragHub

- **What it is:** A unified, standardized meta-database that aggregates multiple open spectral libraries (e.g., MassBank, GNPS, MoNA) into a single, clean schema.
- **Why it matters:** Public libraries use conflicting file formats (`.msp`, `.mgf`, `.json`), inconsistent metadata, and different naming conventions. FragHub harmonizes these sources into a single format with standard InChIKeys and filtered peak lists, making it easy to pull additional experimental training samples if your model needs extra data.

How These Fit Together in a Workflow



User prompt: okay so i want to understand a detail overview of DreamS what does it take as input how the input is process and what is the output

Response: **DreaMS** (**D**eep **R**epresentations **E**mpowering the **A**nnotation of **M**ass **S**pectra) is essentially a **"BERT for mass spectrometry"**.

Before DreaMS, models relied on handcrafted physical fragmentation rules (like fragmentation trees in SIRIUS) or small labeled datasets. DreaMS instead uses a Transformer trained in a self-supervised fashion on tens of millions of raw, unlabeled spectra from **GeMS** (mined from MassIVE/GNPS).

1. What DreaMS Takes as Input

For each MS/MS measurement, DreaMS consumes both the **fragmentation peaks** and **global metadata**:

1. **Peak Sequence:**
   - A variable-length list of peaks sorted by m/z:

$$\text{Peaks} = [(m/z_1, I_1), (m/z_2, I_2), \ldots, (m/z_N, I_N)]$$

   - Typically, spectra are filtered down to the top K most prominent peaks (e.g., $N \leq 60$ to 128) to remove detector baseline noise.
2. **Precursor Context & Neutral Losses:**
   - **Precursor m/z:** The parent ion mass.

- **Neutral loss per peak:** $\Delta m/z_i = m/z_{\text{precursor}} - m/z_i$.
- **Ionization mode / Adduct:** Whether it was collected in positive ($[M+H]^+$) or negative ($[M-H]^-$) polarity.
- **Collision Energy:** The fragmentation energy used during collision-induced dissociation.

2. How the Input is Processed (Under the Hood)

The core architecture is a **Bidirectional Transformer Encoder**:

```
[Raw Spectrum: (m/z, I)]
          │
          ▼
 [Peak Featurization]  ──► Continuous embeddings of (m/z, intensity, Δm/z)
          │
          ▼
   [Transformer]       ──► Multi-Head Self-Attention across all peaks
  (Self-Attention)         (every fragment peak attends to every other peak)
          │
          ▼
 [Readout / Pooling]   ──► Aggregates peak-level states into a single vector
          │
          ▼
 [1024-d DreaMS Vector]
```

Step A: Tokenization & Peak Embedding

- A mass peak is not a discrete word token—it is continuous. DreaMS projects continuous values (m/z, relative intensity I, and neutral loss $\Delta m/z$) into a high-dimensional vector space using sinusoidal positional embeddings or multi-layer perceptrons (MLPs).
- Precursor metadata (charge, parent mass, energy) is injected as a global conditioning prefix (similar to a `[CLS]` token in NLP).

Step B: Self-Attention Across Fragments

- Inside the Transformer layers, **all peaks attend to each other**.
- This allows the model to learn chemical fragmentation pathways without hard-coded rules. For example, if a peak at $m/z = 180$ appears only when a fragment at $m/z = 105$ is present, self-attention captures that co-occurrence relationship.

Step C: How It Was Pretrained (The Self-Supervised Objective)

To understand chemistry without human labels, DreaMS was pretrained using two surrogate objectives on GeMS:

- **Masked Peak Prediction (like BERT's masked tokens):** Random peaks are dropped or masked out. The model must predict their presence, mass, and intensity based on the remaining peaks.
- **Chromatographic Retention Ordering:** Predicting relative LC retention behavior between pairs of runs.

3. What DreaMS Outputs

Depending on how you use the model, it provides two levels of output:

1. **The Core Output: 1024-Dimensional Spectral Embedding**
   - Calling the pretrained encoder produces a dense vector $\mathbf{z} \in \mathbb{R}^{1024}$ per spectrum.
   - This vector is a condensed chemical fingerprint of the spectrum. Two spectra of chemically similar molecules end up close to each other in cosine/Euclidean distance, even if acquired under slightly different instrument tunings.
2. **Downstream Head Outputs (Fine-Tuning / Task-Specific):**
   - **Molecular Fingerprint Prediction:** An MLP classification head on top of the 1024-d embedding that predicts a 2,048-bit binary Morgan/ECFP fingerprint vector.
   - **Spectral Similarity:** Directly comparing two 1024-d vectors to search spectral libraries faster than pairwise peak matching.
   - **De Novo Molecular Decoders:** The 1024-d embedding can act as the cross-attention context for an autoregressive decoder generating SMILES tokens.

How to Use DreaMS in This Competition

In the Enveda CASMI workflow, DreaMS eliminates the need to engineer custom 1D CNNs or peak binners:

```
from dreams.api import dreams_embeddings

# 1. Feed your competition spectra
embeddings = dreams_embeddings("train_spectra.mgf")  # Shape: (N, 1024)

# 2. Train a lightweight classification head (PyTorch)
# maps: 1024-d DreaMS embedding  --->  2048-bit Morgan Fingerprint (or 2D structure candidates)
```

By freezing or fine-tuning the DreaMS backbone, your model starts with prior knowledge of how millions of real molecules fragment under mass spectrometry.

User prompt: NEEDED = ['ingest_lib', 'inchikey14', 'normalized_smiles', 'precursor_mz', 'ms2_mzs', 'ms2_normalized_intensities'] casmi_all_df = pd.read_parquet(TRAIN_PATH, columns=NEEDED, filters=[('ingest_lib', '!=', 'enveda-180')]) now i am starting to work with the data here in this codeblock what does the filters column mean

Response: In this code block:

```
filters = [('ingest_lib', '!=', 'enveda-180')]
```

The `filters` argument tells Pandas and PyArrow to **drop any row where the source library (`ingest_lib`) is `"enveda-180"` before loading the file into RAM**.

1. How It Works Mechanically (Predicate Pushdown)

`train.parquet` is roughly 2.5 million rows and several gigabytes in size.

If you load the entire dataset and run:

```
df = df[df['ingest_lib'] != 'enveda-180']
```

Python must first read the entire multi-gigabyte file into memory, which can easily crash a standard RAM environment (OOM).

Passing `filters=[('ingest_lib', '!=', 'enveda-180')]` directly into `pd.read_parquet()` performs **predicate pushdown**:

- The Parquet storage engine reads the row-group metadata on disk.
- It skips any chunks matching `'enveda-180'` entirely.
- Only the remaining libraries are decompressed and loaded into Python.

2. Why the Tutorial Excludes `'enveda-180'`

The training dataset aggregates multiple spectral libraries, tracked by the `ingest_lib` column:

- **`enveda-180` is huge:** It accounts for **1,153,785 spectra** (almost half of the entire ~2.5 million spectrum dataset).
- **Speed and Kaggle Session Limits:** Loading and training an autoregressive de novo model (like the tutorial's Transformer/BPE model) across 2.5M spectra takes many hours. Excluding `enveda-180` immediately cuts the dataset size in half (~1.39M spectra remain), allowing the notebook to run, tokenize, and train within a standard Kaggle runtime session.
- **Separation of Source Types:** `enveda-180` is a specific synthetic/in-house library released by Enveda. Some developers exclude it to train purely on public reference standards, or hold it aside as a dedicated benchmark/retrieval set.

---

*User prompt: # process data ## spectrum transform def process_spectra(df): def _process_spectrum(row): # sort mzs, intensities sort_mask = np.argsort(row.ms2_normalized_intensities)[::-1] sorted_mzs = row.ms2_mzs[sort_mask] sorted_intensities = row.ms2_normalized_intensities[sort_mask] # truncate to max len truncated_mzs = sorted_mzs[:MAX_LEN - 1] # note: we use MAX_LEN - 1 to account for the precursor, which will be prepended truncated_intensities = sorted_intensities[:MAX_LEN - 1] # normalize intensities normalized_intensities = truncated_intensities / truncated_intensities.max() return truncated_mzs, normalized_intensities mzs, ints = zip(*df.progress_apply(_process_spectrum, axis=1)) df['processed_mzs'] = mzs df['processed_intensities'] = ints return df ## transform spectra print('processing spectra...') casmi_train_df = process_spectra(casmi_train_df) casmi_val_df = process_spectra(casmi_val_df) ## smiles bpe tokenizer def train_bpe_tokenizer(structures): os.environ["TOKENIZERS_PARALLELISM"] = "true" # turn on tokenizers parallelism to speed up bpe training full_alphabet = list(set(list("".join(structures)))) tokenizer = Tokenizer(models.BPE()) special_tokens = [("<pad>", BPE_PAD_ID), ("<s>", BPE_BOS_ID), ("</s>", BPE_EOS_ID),] tokenizer.post_processor = TemplateProcessing(single="<s> $A </s>", special_tokens=special_tokens) special_tokens = ["<pad>", "<s>", "</s>"] trainer = trainers.BpeTrainer(vocab_size=VOCAB_SIZE, initial_alphabet=full_alphabet, special_tokens=special_tokens, show_progress=True,) train_iterator = structures tokenizer.train_from_iterator(train_iterator, trainer) os.environ["TOKENIZERS_PARALLELISM"] = "false" # turn off tokenizers parallelism return tokenizer def tokenize_df_smiles(df, tokenizer): df['bpe_tokenized_smiles'] = df.normalized_smiles.progress_apply(lambda x: np.array(tokenizer.encode(x).ids, dtype=int)) return df ## tokenize smiles print('training bpe...') bpe_train_structures = list(casmi_train_df.normalized_smiles.unique()) tokenizer = train_bpe_tokenizer(bpe_train_structures) print('processing smiles...') casmi_train_df = tokenize_df_smiles(casmi_train_df, tokenizer=tokenizer) casmi_val_df = tokenize_df_smiles(casmi_val_df, tokenizer=tokenizer) # utility for decoding bpe smiles def decode_tokenized_smiles(bpe_smiles_tokens, tokenizer=tokenizer): return tokenizer.decode(bpe_smiles_tokens).replace(' ', '') explain how this codeblock works step by step simply what is it doing and how it is doing tokenization*

Response: This code block prepares raw data for an **Encoder-Decoder (Seq2Seq)** model (similar to an image-captioning or translation model), where the **input** is the mass spectrum and the **output** is the SMILES sequence.

It handles two distinct tasks:

1. **Spectrum Transformation:** Cleans, sorts, and caps the continuous numerical peak lists.
2. **SMILES BPE Tokenization:** Trains a subword tokenizer on chemical text and converts SMILES strings into numeric integer token IDs for autoregressive neural network training.

Part 1: Spectrum Processing (`process_spectra`)

A raw spectrum might contain hundreds of noisy, low-intensity background peaks. This section filters the spectrum down to the most informative signals.

```
# 1. Sort by intensity descending
sort_mask = np.argsort(row.ms2_normalized_intensities)[::-1]
sorted_mzs = row.ms2_mzs[sort_mask]
sorted_intensities = row.ms2_normalized_intensities[sort_mask]
```

- `np.argsort(...)[::-1]` finds the indices of the peaks ordered from **tallest to shortest**.
- Both `m/z` and `intensity` arrays are reordered so the most chemically significant fragments appear first.

```
# 2. Truncate to fixed length
truncated_mzs = sorted_mzs[:MAX_LEN - 1]
truncated_intensities = sorted_intensities[:MAX_LEN - 1]
```

- Variable-length lists are clipped to at most `MAX_LEN - 1` peaks.
- Why `MAX_LEN - 1`? Mass spec models typically prepend the **precursor ion peak** (the parent molecule mass) at position index 0, bringing the total length to `MAX_LEN`.

```
# 3. Re-normalize
normalized_intensities = truncated_intensities / truncated_intensities.max()
```

- After slicing, the intensities are divided by the largest remaining peak so the top peak is guaranteed to equal `1.0`.

Part 2: Byte-Pair Encoding (BPE) Tokenization

Standard language models split text by spaces. However, SMILES strings have **no spaces** (e.g., `CC(=O)Oc1ccccc1C(=O)O` for aspirin).

Instead of treating every single character as a token (which creates long sequences) or every molecule as a whole word (which creates an infinite vocabulary), **Byte-Pair Encoding (BPE)** finds frequent recurring chemical building blocks (rings, functional groups, alkyl chains) and merges them into subword tokens.

1. Setup and Vocabulary Initialization

```
full_alphabet = list(set(list("".join(structures))))
```

- Gathers every unique single character present in the entire training set (letters like `C`, `N`, `O`, `c`, `n`, symbols like `=`, `(`, `)`, `#`, numbers `1`, `2`, etc.). This guarantees the tokenizer can never encounter an "out-of-vocabulary" (OOV) character.

```
special_tokens = [
    ("<pad>", BPE_PAD_ID), # Padding token for batch alignment
    ("<s>", BPE_BOS_ID),   # Beginning-of-sequence token
    ("</s>", BPE_EOS_ID),  # End-of-sequence token
]
```

- Defines control tokens required for sequence-to-sequence autoregressive generation:
  - `<s>`: Prompt given to the decoder to trigger the first molecule token.
  - `</s>`: Predicts when the molecule is complete and halts generation.
  - `<pad>`: Fills batches to uniform length.

2. How the BPE Algorithm Learns Chemical Subwords

When `tokenizer.train_from_iterator(train_iterator, trainer)` runs:

- **Initial State:** Every molecule is split into individual characters: `['C', 'C', '(', '=', 'O', ')']`.
- **Iterative Merging:** The algorithm counts the most frequently adjacent character pairs across all SMILES in the dataset:
  - If `C` and `C` frequently occur together, it creates a new token `CC`.

- If `c1ccccc1` (a benzene ring) frequently appears, it merges stepwise: `c` + `1` → `c1`, `c1` + `c` → `c1c` ... → `c1ccccc1`.

- It repeats these merges until reaching the exact vocabulary capacity defined by `VOCAB_SIZE`.

3. Post-Processing Template

```
tokenizer.post_processor = TemplateProcessing(
    single="<s> $A </s>", special_tokens=special_tokens
)
```

- Wraps every encoded SMILES string automatically with start and stop markers:

$$\text{SMILES} \longrightarrow [\texttt{<s>},\ \text{token}_1,\ \text{token}_2,\ \ldots,\ \text{token}_k,\ \texttt{</s>}]$$

4. Applying to DataFrames (`tokenize_df_smiles`)

```
df['bpe_tokenized_smiles'] = df.normalized_smiles.progress_apply(
    lambda x: np.array(tokenizer.encode(x).ids, dtype=int)
)
```

- Converts each string into an array of integers (token IDs):
  - Input: `"CCO"`
  - Output: `np.array([1, 45, 12, 2])` (where `1` = `<s>`, `45` = `CC`, `12` = `O`, `2` = `</s>`).

Part 3: Decoding Utility (`decode_tokenized_smiles`)

```
def decode_tokenized_smiles(bpe_smiles_tokens, tokenizer=tokenizer):
    return tokenizer.decode(bpe_smiles_tokens).replace(' ', '')
```

- When the trained model predicts an array of integer IDs during test-time inference, this function maps the integer IDs back to subword strings, concatenates them, and strips internal tokenizer delimiter spaces (`replace(' ', '')`) to produce a valid SMILES string.

User prompt: # define tanimoto similarity for metrics ## we'll use Morgan radius 2 fps to define "tanimoto similarity" in this notebook ### note: radius 2, counts=True, sparse (no hashing) fp_gen = rdFingerprintGenerator.GetMorganGenerator(radius=2) def tanimoto_smiles(smiles1, smiles2): mol1, mol2 = MolFromSmiles(smiles1), MolFromSmiles(smiles2) if mol1 is None or mol2 is None: return None fp1, fp2 = fp_gen.GetSparseFingerprint(mol1), fp_gen.GetSparseFingerprint(mol2) return DataStructs.TanimotoSimilarity(fp1, fp2) explain what is Morgan fingerprint and what is tanimoto similarity and what is happening here

Response: 1. What is a Morgan Fingerprint?

Computers cannot directly run vector math on a chemical structure. A **molecular fingerprint** converts a 2D chemical graph into a numerical representation (a bit vector or count vector) describing which sub-structures are present.

The **Morgan Fingerprint** (an implementation of Extended-Connectivity Fingerprints, or **ECFP**) builds this representation in expanding concentric circles around every atom:

- **Radius 0:** Looks only at individual atoms (e.g., "Carbon", "Oxygen with a hydrogen attached").
- **Radius 1:** Expands one bond outward, recording each atom and its immediate neighbors (e.g., "Carbon bonded to Carbon and Oxygen").
- **Radius 2 (equivalent to ECFP4):** Expands two bonds outward, capturing functional groups, rings, and branched motifs.

```
Radius 0:        Radius 1:              Radius 2:
[Atom]    ⟶   [Atom + Neighbors]  ⟶ [Local Functional Group]
 ( C )            ( C - O )             ( C - C(=O) - OH )
```

As the radius expands, each unique local atomic environment is hashed into an identifier.

2. What is Tanimoto Similarity?

Tanimoto similarity (the Jaccard index for chemistry) measures the overlap between two molecular fingerprints. It produces a score between **0.0** (completely distinct structures, zero shared fragments) and **1.0** (identical sub-structural fragments).

- **Set / Bit form:**

$$\text{Tanimoto}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{c}{a + b - c}$$

  Where:

  - a = number of features present in Molecule A

- b = number of features present in Molecule B
- c = number of features shared by both

- **Count / Vector form (used in this code):** When counting how many times each fragment appears, it computes the min-max intersection over union:

$$\text{Tanimoto}(\mathbf{u}, \mathbf{v}) = \frac{\sum_k \min(u_k, v_k)}{\sum_k \max(u_k, v_k)}$$

If Molecule A has three benzene rings and Molecule B has one, the count version accounts for that difference rather than treating it as a simple binary yes/no.

3. Step-by-Step Code Walkthrough

```
fp_gen = rdFingerprintGenerator.GetMorganGenerator(radius=2)
```

- Initializes RDKit's modern fingerprint generator with **`radius=2`** (capturing fragments up to 2 bonds away from every center atom).

```
def tanimoto_smiles(smiles1, smiles2):
    mol1, mol2 = MolFromSmiles(smiles1), MolFromSmiles(smiles2)
    if mol1 is None or mol2 is None:
        return None
```

- Converts two raw SMILES strings into RDKit molecule objects.
- If either string is chemically invalid (e.g., bad valence, unclosed rings generated by a neural network), `MolFromSmiles` returns `None`, and the function gracefully returns `None` instead of crashing.

```
    fp1, fp2 = fp_gen.GetSparseFingerprint(mol1), fp_gen.GetSparseFingerprint(mol2)
```

- **`GetSparseFingerprint`:** Standard Morgan fingerprints fold identifiers into a fixed-length bit array (like 1,024 or 2,048 bits) using modulo hashing, which causes "hash collisions" (two distinct fragments sharing the same bit).
- A **sparse fingerprint** avoids bit collisions: it preserves the exact 32-bit/64-bit integer IDs of every sub-structure and retains the **frequency counts** of how many times each fragment appears.

```
    return DataStructs.TanimotoSimilarity(fp1, fp2)
```

- Calculates the count-based Tanimoto coefficient between the two sparse vectors and returns a float from `0.0` to `1.0`.

Why This is Used in the Competition

While the official competition metric is strict 0-or-1 InChIKey14 matching (MRR@25), Tanimoto similarity serves as a **continuous evaluation metric** during model development:

- If your model's predicted SMILES is not an exact match to the true molecule, exact string or InChIKey matching scores a flat `0.0`.
- Tanimoto similarity reveals whether the model predicted an isomer or close analog (e.g., `0.85` similarity means the model correctly deduced most of the molecular skeleton from the mass spectrum, even if one functional group shifted position).

---

*User prompt: # define module classes ### peak embedder class PeakEmbedder(nn.Module): """embed (m/z, intensity) peak pairs with sinusoidal embeddings from Voronov et al""" def __init__(self, d_model, dropout, sin_dim=None, mz_log_lims=(-2., 3.), mz_log_power=1.0): super().__init__() sin_dim = sin_dim if sin_dim is not None else d_model self.dropout = dropout wavelength = torch.pow(10, (mz_log_lims[1] - mz_log_lims[0]) * torch.pow(torch.linspace(0, 1, int(sin_dim / 2)), mz_log_power,) + mz_log_lims[0],) frequency = 2 * np.pi / wavelength self._frequency = nn.Parameter(frequency, requires_grad=False) self._ff_block_1 = nn.Sequential(*[nn.Linear(sin_dim, d_model), nn.ReLU(), nn.Dropout(dropout), nn.Linear(d_model, d_model), nn.Dropout(dropout),]) self._ff_block_2 = nn.Sequential(*[nn.Linear(d_model+1, d_model), nn.ReLU(), nn.Dropout(dropout), nn.Linear(d_model, d_model), nn.Dropout(dropout),]) def forward(self, mz_tensor, intensity_tensor): # "mz embedding" embeds just the m/z value omega_mz = self._frequency.view(*(1 for _ in range(mz_tensor.ndim)), -1) * mz_tensor.unsqueeze(-1) sin = torch.sin(omega_mz) cos = torch.cos(omega_mz) mz_vecs = torch.cat([sin, cos], dim=-1) mz_embeds = self._ff_block_1(mz_vecs) # "peak embedding" embeds both the m/z and intensity peak_embeds = torch.cat([mz_embeds, intensity_tensor.unsqueeze(2)], dim=2) return self._ff_block_2(peak_embeds) ### spectrum encoder class SpectrumEncoder(nn.Module): def __init__(self, embed_dim, n_heads, n_layers, dim_feedforward=None, dropout=0.1, activation='relu'): super().__init__() dim_feedforward = dim_feedforward if dim_feedforward is not None else 4 * embed_dim self.encoder = TransformerEncoder(TransformerEncoderLayer(embed_dim, n_heads, dim_feedforward=dim_feedforward, batch_first=True, dropout=dropout, activation=activation,), n_layers,) self.init_weights() def forward(self, sequence_input, attention_mask): pad_mask = (attention_mask == 0) return self.encoder(sequence_input, src_key_padding_mask=pad_mask) def init_weights(self): for layer in self.encoder.layers: for name, mod in layer.named_modules(): if isinstance(mod, nn.Linear): nn.init.xavier_uniform_(mod.weight) if mod.bias is not None: nn.init.constant_(mod.bias, 0.0) elif isinstance(mod, nn.LayerNorm): nn.init.constant_(mod.weight, 1.0) if mod.bias is not None: nn.init.constant_(mod.bias, 0.0) ### smiles decoder class SmilesDecoder(nn.Module): def __init__(self, embed_dim, vocab_size, n_layers, n_heads, pad_token_id=1, bos_token_id=0, eos_token_id=2, dim_feedforward=None, dropout=0.1, activation='gelu', validate_n_samples=10, predict_n_samples=25): super().__init__() self.bos_token_id = bos_token_id self.pad_token_id = pad_token_id self.eos_token_id = eos_token_id*

*self.vocab_size = vocab_size self.n_layers = n_layers self.n_heads = n_heads self.embed_dim = embed_dim dim_feedforward = dim_feedforward if dim_feedforward is not None else 4 * self.embed_dim self.n_samples_per_stage = {'validate': validate_n_samples, 'predict': predict_n_samples} self.wte = nn.Embedding(vocab_size, self.embed_dim, padding_idx=self.pad_token_id) self.decoder = nn.TransformerDecoder(decoder_layer=nn.TransformerDecoderLayer(d_model=self.embed_dim, dim_feedforward=dim_feedforward, nhead=self.n_heads, dropout=dropout, activation=activation, batch_first=True,), num_layers=self.n_layers,) self.lm_head = nn.Linear(self.embed_dim, vocab_size, bias=False) self.init_weights() def init_weights(self): torch.nn.init.zeros_(self.lm_head.weight) # zero out classifier weights to start torch.nn.init.normal_(self.wte.weight, mean=0.0, std=1.0) # if self.wte.weight.device.type == "cuda": # save memory by casting embeddings to bf16 # self.wte.to(dtype=torch.bfloat16) for layer in self.decoder.layers: for name, mod in layer.named_modules(): if isinstance(mod, nn.Linear): nn.init.xavier_uniform_(mod.weight) if mod.bias is not None: nn.init.constant_(mod.bias, 0.0) elif isinstance(mod, nn.LayerNorm): nn.init.constant_(mod.weight, 1.0) if mod.bias is not None: nn.init.constant_(mod.bias, 0.0) def forward(self, idx, encoder_outputs, encoder_attention_mask, structure_tokens=None): encoder_pad_mask = (encoder_attention_mask == 0) softcap = 15 tgt = self.wte(idx) if structure_tokens is not None: tgt_mask = nn.Transformer.generate_square_subsequent_mask(tgt.shape[1]) else: tgt_mask = None x = self.decoder(tgt=tgt, memory=encoder_outputs, tgt_mask=tgt_mask, memory_key_padding_mask=encoder_pad_mask,) logits = self.lm_head(x) logits = softcap * torch.tanh(logits / softcap) logits = logits.float() output_dict = { 'logits': logits, } if structure_tokens is not None: loss = F.cross_entropy(logits.view(-1, logits.size(-1)), structure_tokens.view(-1), ignore_index=self.pad_token_id, reduction='mean') output_dict['loss'] = loss return output_dict @torch.inference_mode() def generate(self, encoder_states, encoder_mask, n_samples=1, max_new_tokens=50, temperature=1.0,): """simple sampling based generation with temperature control""" B, S, D = encoder_states.shape idx = torch.ones_like(encoder_states[:, :1, 0]).long() * self.bos_token_id logprob = torch.zeros_like(encoder_states[:, :1, 0]) idx = idx.expand(-1, n_samples).reshape(B * n_samples, -1) idx_next = idx logprob_expanded = logprob.expand(-1, n_samples).reshape(B * n_samples, -1) eos_generated = idx[:, -1] == self.eos_token_id encoder_states = (encoder_states.unsqueeze(1) .expand(-1, n_samples, -1, -1) .reshape(1, B * n_samples, S, D) .squeeze(0)) encoder_mask = (encoder_mask.unsqueeze(1) .expand(-1, n_samples, -1) .reshape(1, B * n_samples, S) .squeeze(0)) for _ in range(max_new_tokens): # forward the model to get the logits for the index in the sequence logits = self.forward(idx_next, encoder_states, encoder_mask, structure_tokens=None,)['logits'] # pluck the logits at the final step and scale by desired temperature rescaled_logits = (logits[:, -1, :] / temperature).log_softmax(dim=-1) logits = logits[:, -1, :].log_softmax(dim=-1) if eos_generated.sum() > 0: logits[eos_generated, :] = -float("Inf") logits[eos_generated, self.eos_token_id] = 0 rescaled_logits[eos_generated, :] = -float("Inf") rescaled_logits[eos_generated, self.eos_token_id] = 0 # sample from the distribution idx_next = torch.multinomial(rescaled_logits.softmax(dim=-1), num_samples=1) token_logits_next = torch.take_along_dim(logits, idx_next, dim=1) # append sampled index to the running sequence and continue idx = torch.cat((idx, idx_next), dim=1) logprob_expanded = torch.cat((logprob_expanded, token_logits_next), dim=1) eos_generated = idx[:, -1] == self.eos_token_id if eos_generated.sum() == len(eos_generated): break if (idx == self.eos_token_id).max(axis=-1).values.sum() == idx.shape[0]: break _, L = idx.shape return idx.reshape(B, n_samples, L), logprob_expanded.sum(dim=-1).reshape(B, n_samples) analyse and explain tihs code block step by step*

Response: This code defines the core **Sequence-to-Sequence (Seq2Seq)** neural network architecture that translates mass spectra into SMILES strings. It consists of three decoupled components:

1. **`PeakEmbedder`**: Turns continuous $(m/z, \text{intensity})$ values into dense vectors using sinusoidal frequencies.
2. **`SpectrumEncoder`**: A standard bidirectional Transformer encoder that models interactions across all peaks.
3. **`SmilesDecoder`**: An autoregressive Transformer decoder that attends to the encoder representations and generates SMILES tokens via temperature-controlled sampling.

1. `PeakEmbedder`: Embedding Continuous Physical Quantities

Standard NLP embeddings map discrete token IDs to vectors via lookup tables (`nn.Embedding`). Mass spectrometry data consists of **continuous numbers** ($m/z \in [0, 2000]$, $\text{intensity} \in [0, 1]$), which cannot use lookup tables.

```
wavelength = torch.pow(
    10,
    (mz_log_lims[1] - mz_log_lims[0]) * torch.pow(
        torch.linspace(0, 1, int(sin_dim / 2)),
        mz_log_power,
    ) + mz_log_lims[0],
)
frequency = 2 * np.pi / wavelength
self._frequency = nn.Parameter(frequency, requires_grad=False)
```

- **Sinusoidal Encoding (Voronov et al.):** Borrowed from positional encodings in the original Transformer paper, but mapped over a **logarithmic scale** (from $10^{-2} = 0.01$ Da to $10^{3} = 1000$ Da).
- Logarithmic spacing ensures fine-grained frequency resolution at low masses (where 0.01 Da differences matter) while covering large fragment ranges.
- `requires_grad=False` freezes these frequencies as static base transformations.

```
def forward(self, mz_tensor, intensity_tensor):
    omega_mz = self._frequency.view(...) * mz_tensor.unsqueeze(-1)
    sin = torch.sin(omega_mz)
    cos = torch.cos(omega_mz)
    mz_vecs = torch.cat([sin, cos], dim=-1)
    mz_embeds = self._ff_block_1(mz_vecs)
```

```
    peak_embeds = torch.cat([mz_embeds, intensity_tensor.unsqueeze(2)], dim=2)
    return self._ff_block_2(peak_embeds)
```

1. Multiplies the m/z scalar by the frequency bank to get phases: $\omega \cdot (m/z)$.
2. Passes them through $[\sin, \cos]$ pairs to create high-dimensional continuous representations.
3. Projects the sinusoidal vector through an initial 2-layer MLP (`_ff_block_1`).
4. Concatenates the scalar peak intensity (`dim=2`) directly onto the mass vector: `[mz_vector, intensity]`.
5. Projects the merged vector through a second MLP (`_ff_block_2`) to produce a single unified peak embedding of dimension `d_model`.

2. `SpectrumEncoder`: Modeling Peak-to-Peak Co-occurrence

```
self.encoder = TransformerEncoder(
    TransformerEncoderLayer(
        embed_dim, n_heads, dim_feedforward=dim_feedforward,
        batch_first=True, dropout=dropout, activation=activation,
    ),
    n_layers,
)
```

- Takes the embedded peak sequence of shape `(Batch, Sequence_Length, embed_dim)`.
- Uses standard **bidirectional self-attention**: every peak attends to every other peak in the spectrum. This models chemical fragmentation rules (e.g., peak A and peak B breaking together from the same substructure).

```
def forward(self, sequence_input, attention_mask):
    pad_mask = (attention_mask == 0)
    return self.encoder(sequence_input, src_key_padding_mask=pad_mask)
```

- Mass spectra have different numbers of peaks. The shorter ones are padded with zeros.
- `pad_mask = (attention_mask == 0)` tells PyTorch's Transformer to zero out attention weights for padding tokens so they do not contaminate the true spectral signals.
- Weights are initialized with Xavier uniform initialization for linear layers and constant values for LayerNorm.

3. `SmilesDecoder`: Architecture and Training Forward Pass

The decoder is a causal (autoregressive) Transformer decoder with a vocabulary projection head (`lm_head`).

```
self.wte = nn.Embedding(vocab_size, self.embed_dim, padding_idx=self.pad_token_id)
self.lm_head = nn.Linear(self.embed_dim, vocab_size, bias=False)
```

- `wte`: Token embedding layer that maps BPE token IDs into vectors.
- `lm_head`: Linear projection mapping decoder states back to vocabulary logits.

Forward Pass Mechanisms

```
if structure_tokens is not None:
    tgt_mask = nn.Transformer.generate_square_subsequent_mask(tgt.shape[1])
```

- **Causal Masking:** When training with ground-truth SMILES (`structure_tokens`), a lower-triangular causal mask (`tgt_mask`) ensures that step t can only attend to previous tokens $< t$, preventing the model from looking ahead.

```
x = self.decoder(
    tgt=tgt,
    memory=encoder_outputs,
    tgt_mask=tgt_mask,
    memory_key_padding_mask=encoder_pad_mask,
)
logits = self.lm_head(x)
logits = softcap * torch.tanh(logits / softcap)
```

- **Cross-Attention (`memory=encoder_outputs`):** The SMILES decoder queries the peak embeddings generated by `SpectrumEncoder`.
- **Logit Soft-Capping (`softcap = 15`):** Instead of allowing unbounded logits that cause gradient spikes and overconfident/collapsed probability distributions, it applies:

$$\text{logits}_{\text{capped}} = 15 \cdot \tanh\left(\frac{\text{logits}}{15}\right)$$

This bounds all logits strictly to the interval $(-15, +15)$, improving training stability.

```
if structure_tokens is not None:
    loss = F.cross_entropy(
        logits.view(-1, logits.size(-1)),
        structure_tokens.view(-1),
        ignore_index=self.pad_token_id,
        reduction='mean'
    )
    output_dict['loss'] = loss
```

- Computes Cross-Entropy loss over the vocabulary, masking out `pad_token_id`.

4. `generate`: Autoregressive Sampling for Inference

The `@torch.inference_mode()` generator produces candidate SMILES strings for test spectra.

```
idx = idx.expand(-1, n_samples).reshape(B * n_samples, -1)
encoder_states = encoder_states.unsqueeze(1).expand(-1, n_samples, -1, -1).reshape(...)
```

- **Batch Expansion for Multiple Candidates:** The competition evaluates **MRR@25**, requiring up to 25 ranked guesses per spectrum.
- This duplicates each spectrum B into $B \times n_samples$ parallel rows, enabling parallel generation of 25 distinct SMILES candidates per spectrum in a single forward loop.

Inside the Generation Loop (`for _ in range(max_new_tokens)`)

1. **Forward Pass:** Generates logits for the current partial SMILES sequence.
2. **Temperature Scaling:**

   ```
   rescaled_logits = (logits[:, -1, :] / temperature).log_softmax(dim=-1)
   ```

   - Diving by `temperature` controls randomness ($T < 1.0$ makes sampling more deterministic/conservative; $T > 1.0$ produces more diverse chemical exploration).
3. **EOS Locking:**

   ```
   if eos_generated.sum() > 0:
       logits[eos_generated, :] = -float("Inf")
       logits[eos_generated, self.eos_token_id] = 0
   ```

   - Once a candidate predicts the End-Of-Sequence token (`</s>`), it is locked into generating only `</s>` with probability 1.0 until all parallel sequences finish.
4. **Multinomial Sampling:**

   ```
   idx_next = torch.multinomial(rescaled_logits.softmax(dim=-1), num_samples=1)
   ```

   - Samples the next token from the predicted probability distribution.
5. **Tracking Log-Probabilities:**
   - It logs `token_logits_next` at every step, summing them over the sequence. This cumulative log-probability provides a **confidence score** used to rank the generated molecules from #1 to #25 for the final submission.
6. **Termination:** Breaks early if every sequence across the batch has emitted the `eos_token_id`.

User prompt: 2 — known structure, no public spectraNo public spectra exist for this molecule, but the structure is in PubChem or the natural-products database COCONUT, so it is reachable by database retrieval. so i want to understand about the COCONUT dataset what does it contains explain simply

Response: **COCONUT** stands for the **COmprehensive NAtural ProdUcts DataBTH** (Collection of Open Natural Products).

It is an open-access, curated database dedicated specifically to **natural products**—chemical compounds produced by living organisms such as plants, fungi, bacteria, marine life, and animals.

What COCONUT Contains

1. **Chemical Structures (Over 400,000 Compounds):**
   - High-quality curated representations of natural molecules, stored as **SMILES**, **InChI**, and **InChIKey** identifiers.
   - Standard 2D molecular graphs with defined atom connectivities, aromatic rings, and functional groups.

2. **Physicochemical Properties:**
    - Exact monoisotopic masses and calculated molecular weights.
    - Elemental molecular formulas (e.g., $\mathrm{C_{20}H_{24}O_7}$).
    - Standard molecular descriptors: hydrogen bond donors/acceptors, topological polar surface area (TPSA), and $\log P$.
3. **Biological & Taxonomic Annotations:**
    - **Source organisms:** Which plant species, bacterial strain, or fungus produces the molecule.
    - **Taxonomic classification:** Kingdom, family, genus, and geographic/environmental origins.
    - **Natural product chemical taxonomy:** Classification into families like alkaloids, terpenoids, flavonoids, polyketides, and steroids.
4. **Cross-Database References:**
    - Direct links mapping each natural product to PubChem CIDs, ChEMBL IDs, and relevant scientific literature.

Why COCONUT Matters for "Class 2" in This Competition

The competition notes that **Novelty Class 2** consists of molecules with:

> *"Known structure, no public spectra: No public spectra exist for this molecule, but the structure is in PubChem or the natural-products database COCONUT, so it is reachable by database retrieval."*

1. Why Mass Spectrometry Needs Natural Product Libraries

Many secondary metabolites extracted from plants or fungi have known chemical structures published in old academic papers or natural product surveys, but nobody has ever uploaded a modern, public LC-MS/MS spectrum to repositories like GNPS or MassBank.

2. The Retrieval Strategy

Because no spectrum exists in your training set to "memorize," an end-to-end matching lookup fails. However, because the **2D structure** is cataloged in COCONUT:

- You extract the precursor ion mass from your test spectrum (e.g., exact mass $348.121\ \mathrm{Da}$).
- You query the COCONUT database for candidate structures within a tight mass tolerance (e.g., $\pm 10\ \mathrm{ppm}$).
- You rank those candidate structures using your model's predicted sub-structural fingerprints or fragment matching scores.

This allows your model to predict the exact correct InChIKey14 without having to generate novel SMILES completely from scratch.

User prompt: what i have understood that it is saying here this competition is providing the spectra for a compound that does not exist so we have spectra in test set using that spectra we have to predict the smiles now in that case this coconut database could help us now analyse and explain simply if my understanding is correct or not Now in this case how the coconut dataset will help us explain simply

Response: Your understanding is **partially correct, but with one key clarification** regarding what "does not exist" actually means.

Clarifying Your Understanding: What Actually Exists?

- **What you said:** *"The competition is providing the spectra for a compound that does not exist."*
- **The reality:** The compound **does physically exist in the real world**—a real chemical sample was placed into the Bruker timsTOF machine and fragmented to produce that spectrum.
- What **does not exist** is a **publicly available spectrum in open training databases**.

Nobody previously uploaded a reference spectrum of this molecule to open repositories like GNPS or MassBank. Therefore, a model cannot simply "look up" the spectrum like a barcode in a store database.

The Three Novelty Classes & Where COCONUT Fits

The test set categorizes molecules into three novelty classes to test different levels of generalization:

Class	Molecule Status	Public Spectrum Exists?	Chemical Structure Exists in Databases?	Does COCONUT Help?
Class 1	Known compound, known spectrum	Yes	Yes (in PubChem/COCONUT)	Yes (Direct library retrieval)

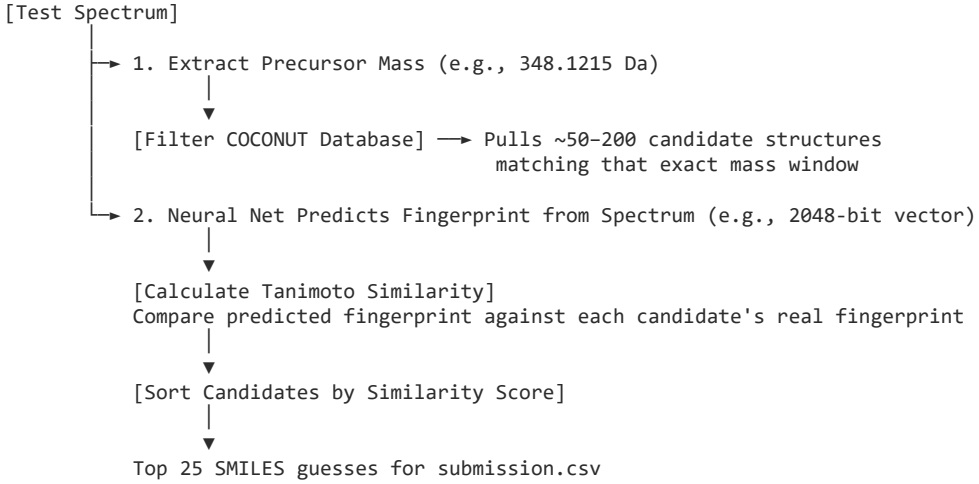
Class	Molecule Status	Public Spectrum Exists?	Chemical Structure Exists in Databases?	Does COCONUT Help?
Class 2	**Known compound, NO public spectrum**	**No**	**Yes (in COCONUT / PubChem)**	**Major Target: Huge impact**
Class 3	Completely novel compound	No	No (brand new discovery)	No (Requires de novo generation)

Your understanding is spot on for **Class 2**.

How COCONUT Helps You Step-by-Step

In Class 2, the spectrum is new to your model, but the molecule's **2D structure is already cataloged inside COCONUT**.

Instead of asking a generative language model to imagine a valid chemical string character-by-character from scratch, you turn the problem into a **candidate search and ranking problem**:



Step 1: Drastically Narrowing the Search Space by Mass

The high-resolution timsTOF instrument gives you the precursor ion's exact monoisotopic mass to 4 decimal places.

- Searching all possible chemistry produces billions of theoretical structures.
- Filtering COCONUT within a narrow ± 10 ppm mass window instantly narrows the candidates down from hundreds of thousands to just **tens or a few hundred real natural product molecules**.

Step 2: Ranking the Candidates

Your neural network (or a model like DreaMS) examines the fragment peaks in the test spectrum and predicts which chemical sub-structures exist in the molecule (e.g., "contains a benzene ring, an ester bond, and a methoxy group").

- You compute the similarity between your predicted substructures and the true substructures of each candidate pulled from COCONUT.
- The best-matching candidate from COCONUT becomes guess #1, the second-best becomes guess #2, up to guess #25.

Step 3: Guaranteed Chemical Validity

Generating SMILES from scratch with an autoregressive Transformer often produces invalid valences or non-existent chemical rings. Picking real molecules directly from COCONUT guarantees that every single guess is a **100% chemically valid, real-world natural product**.

User prompt: 4.3 novelity classes 1. both compound smiles and spectra exists completely doable as a search problem matching the spectra with known spectra normal models like spec2spec converting spectrum to vector then comparison can work here 2. only compound is in coconut no available spectra can reduced to a search problem Our model will predict Morgan finger print then we have two filters one is mass using mass filter compounds second using tanimoto similarity or some similarity based methods find which compound matches best with our predicted morgan finger print Good Search can tackle this class 3. novel both spectra and compound is not in public so after model predicts morgan fingerprint a model is need to predict smiles from morgan finger print or something like that then rank those here i have provided my understanding of this 3 novelity class and how can those be tackled now critically evaluate my understanding and explain if it is correct or not and where is the error

Response: Your overall mental model is largely on the right track, particularly for Class 1 and Class 2. However, there is a **fundamental flaw in your Class 3 strategy** (predicting SMILES from a predicted Morgan fingerprint), along with a few dangerous blind spots in Class 1 and Class 2.

Here is a critical evaluation of each class, pointing out what is correct, what is broken, and how to fix the errors.

Class 1: Known Compound & Known Public Spectrum

Your Understanding:

> *"Both compound smiles and spectra exists completely doable as a search problem matching the spectra with known spectra normal models like spec2spec converting spectrum to vector then comparison can work here"*

The Verdict: Mostly Correct, but with an Overlooked Trap

- **Where you are right:** This is indeed a classical spectral library matching problem. Mapping spectra into vectors (via cosine similarity, modified cosine in `matchms`, or dense neural representations like DreaMS embeddings) and running nearest-neighbor search against the training libraries works reliably.
- **The Error / Blind Spot:**
  1. **Instrument Variability:** The test set was acquired on a **Bruker timsTOF**. The public spectra in your training set come from dozens of different instruments (Orbitraps, Q-TOFs, ion traps) at varying collision energies. A raw spectral vector comparison often fails if the model overfits to the instrument-specific peak intensity ratios.
  2. **Adduct Shifts:** You cannot do a flat vector search without aligning precursor ions first. If your test spectrum is $[M + Na]^+$ and the reference library only has $[M + H]^+$, direct cosine similarity drops to zero unless you account for precursor mass shifts and neutral losses.

Class 2: Known Structure in Databases, No Public Spectrum

Your Understanding:

> *"Only compound is in coconut no available spectra can reduced to a search problem. Our model will predict Morgan finger print then we have two filters one is mass using mass filter compounds second using tanimoto similarity or some similarity based methods find which compound matches best with our predicted morgan finger print Good Search can tackle this class"*

The Verdict: Highly Accurate Strategy (The Industry Standard for CASMI)

- **Where you are right:** This is the exact foundational design of top-tier tools like **CSI:FingerID (SIRIUS)** and **MIST**. Pre-filtering a structural database by exact mass, predicting a high-dimensional fingerprint from the spectrum, and ranking candidates via Tanimoto/Platt scoring is mathematically sound.
- **The Nuance & Blind Spot:**
  1. **COCONUT is not the only database:** While the competition mentions COCONUT, it also explicitly notes **PubChem**. Many natural product derivatives and small molecules exist in PubChem or ChEMBL but are not yet cataloged in COCONUT. If your candidate pool is strictly limited to COCONUT, you will miss valid candidates.
  2. **Adduct mass vs. Monoisotopic mass:** You cannot filter candidate molecules directly using `precursor_mz`. You must first hypothesize the adduct (e.g., if it is $[M + H]^+$, subtract 1.0078 Da; if $[M + Na]^+$, subtract 22.989 Da) to obtain the true neutral monoisotopic mass before querying the candidate database.

Class 3: Completely Novel (No Spectrum, No Database Compound)

Your Understanding:

> *"Novel both spectra and compound is not in public so after model predicts morgan fingerprint a model is need to predict smiles from morgan finger print or something like that then rank those"*

The Verdict: Critical Error in Logic

- **Where the Error Lies: Inverting a Morgan Fingerprint back into a valid SMILES is a notoriously difficult, ill-posed inverse problem.**
  - A Morgan fingerprint maps circular neighborhoods around atoms into hashed integers. It intentionally discards global graph topology and geometry.
  - More critically: your predicted fingerprint is an **approximate, noisy guess** from a neural network (many bits will be false positives or false negatives).
  - Trying to feed a noisy, non-physical bit vector into a secondary model to "reconstruct" a valid, connected chemical graph usually produces chemically invalid garbage, broken valence bonds, or fails entirely.

The Correct Way to Tackle Class 3:

You do **not** use a two-step `Spectrum -> Fingerprint -> SMILES` inversion pipeline here. Instead, you have two primary, proven options:

1. **Direct Autoregressive De Novo Generation (`Spectrum -> SMILES`):**
   - This is exactly why the competition provided the tutorial code you looked at earlier (`SpectrumEncoder` + `SmilesDecoder`).
   - The decoder directly learns the grammar of SMILES conditioned on the spectral peak tokens.

- You use **Beam Search** or **Temperature-Controlled Top-k/Nucleus Sampling** with $N = 25$ to autoregressively sample candidate SMILES strings directly from the spectrum, validating them on the fly with RDKit.

2. **Fragment Assembly / Genetic Algorithm Approaches:**
   - Break down known natural product scaffolds into modular building blocks (subgraphs).
   - Assemble valid molecular graphs whose simulated fragments or substructures align best with the observed peaks (similar to how tools like *MSNovelist* operate).

Comparison of the Three Approaches

Novelty Class	Correct Tactical Approach	Why Your Original Conception Needed Adjustment
Class 1	Library Spectral Matching (Cosine / DreaMS distance)	Solid. Just ensure you normalize for adducts ($[M+H]^+$, $[M+Na]^+$) and mass shifts.
Class 2	Candidate Retrieval (Exact Mass Window $\rightarrow$ Fingerprint Tanimoto Ranking)	Strongest concept. Expand the candidate pool beyond just COCONUT to include relevant PubChem/ChEMBL subsets.
Class 3	**End-to-End Seq2Seq Generation** (`Spectrum -> SMILES` via Beam Search)	**Do not attempt Fingerprint $\rightarrow$ SMILES inversion.** Use the Seq2Seq Transformer decoder directly.

A competitive submission typically ensembles these: allocate your top ranked slots to mass-matched database hits (Classes 1 & 2), and backfill remaining slots with high-confidence de novo generated SMILES (Class 3).

User prompt: Adduct Shifts: You cannot do a flat vector search without aligning precursor ions first. If your test spectrum is [M+Na]^+ and the reference library only has [M+H]^+, direct cosine similarity drops to zero unless you account for precursor mass shifts and neutral losses. Uh, I want to understand the adduct part. How does uh using different adduct changes the uh mass spectrometer? Explain step by step with numeric examples simply.

Response: A mass spectrometer cannot measure neutral molecules directly. It can only guide, accelerate, and detect ions that carry an electrical charge.

To give a neutral molecule (M) a charge during electrospray ionization (ESI), the instrument attaches a small ambient charged particle to it. That attached tag is called an **adduct**.

The two most common adducts in positive-mode mass spectrometry are:

- **Protonation ($[M+H]^+$):** The molecule captures a loose hydrogen ion (H^+, mass ≈ 1.0078 Da).
- **Sodiation ($[M+Na]^+$):** The molecule captures a sodium ion (Na^+, mass ≈ 22.9898 Da, present as trace salt from glass vials or solvents).

Step 1: How the Adduct Changes the Precursor Mass

Suppose the biological sample contains **Glucose** ($C_6H_{12}O_6$).

- **True Neutral Monoisotopic Mass (M):** 180.0634 Da

Depending on which ion attaches in the source chamber, the instrument registers two completely different parent masses:

- **If it picks up a Proton ($[M+H]^+$):**

$$\text{Precursor } m/z = 180.0634 + 1.0078 = \mathbf{181.0712}$$

- **If it picks up Sodium ($[M+Na]^+$):**

$$\text{Precursor } m/z = 180.0634 + 22.9898 = \mathbf{203.0532}$$

The instrument reports a precursor mass difference of:

$$203.0532 - 181.0712 = \mathbf{21.9820} \text{ Da}$$

Even though the sample is the exact same molecule, the instrument registers two parent peaks roughly 22 Da apart.

Step 2: What Happens During Fragmentation (MS/MS)

When the collision chamber smashes the ion, fragments that still carry the charge will retain that adduct tag.

Suppose Glucose loses one water molecule ($\mathrm{H_2O} = 18.0106\ \mathrm{Da}$):

- **For the $[M+H]^+$ precursor (181.0712):**

$$\text{Fragment } m/z = 181.0712 - 18.0106 = \mathbf{163.0606}$$

- **For the $[M+Na]^+$ precursor (203.0532):**

$$\text{Fragment } m/z = 203.0532 - 18.0106 = \mathbf{185.0426}$$

Notice that every fragment carrying the sodium ion is shifted by $+21.9820\ \mathrm{Da}$ compared to the protonated fragment.

Step 3: Why Naive Vector Search Fails (The "Zero Similarity" Trap)

Imagine your test spectrum and reference library entry side-by-side:

Peak Type	Test Spectrum ($[M+Na]^+$)	Reference Spectrum ($[M+H]^+$)	Difference
Precursor	203.05	181.07	$+21.98\ \mathrm{Da}$
Fragment 1 ($-\mathrm{H_2O}$)	185.04	163.06	$+21.98\ \mathrm{Da}$
Fragment 2 ($-2\mathrm{H_2O}$)	167.03	145.05	$+21.98\ \mathrm{Da}$

If you convert both spectra into standard m/z binned vectors and calculate a dot product (cosine similarity):

- Index 163 in the reference has intensity, but index 163 in the test set is empty (0).
- Index 185 in the test set has intensity, but index 185 in the reference is empty (0).

Because not a single peak aligns on the absolute m/z axis, the dot product is 0.0. The algorithm declares them completely different compounds, failing to recognize they are the exact same chemical structure under two different adduct states.

Step 4: The Invariant That Fixes It (Neutral Loss)

While the absolute fragment masses shift, the **neutral loss** (the mass difference between the precursor and the fragment) remains identical:

- **In the $[M+H]^+$ spectrum:**

$$\Delta m/z = 181.0712 - 163.0606 = \mathbf{18.0106}\ \mathrm{Da}$$

- **In the $[M+Na]^+$ spectrum:**

$$\Delta m/z = 203.0532 - 185.0426 = \mathbf{18.0106}\ \mathrm{Da}$$

Both spectra show an identical loss of $18.0106\ \mathrm{Da}$ ($\mathrm{H_2O}$).

This is why spectral encoders (like the `PeakEmbedder` discussed earlier, DreaMS, and `matchms`'s Modified Cosine score) do not look solely at raw m/z; they calculate $\Delta m/z = m/z_{\text{precursor}} - m/z_{\text{fragment}}$ so that adduct mass shifts cancel out cleanly.

User prompt: Now with a step-by-step reasoning, explain if we take the neutral loss, do we need to care about what adduct was added or explain that simply or there is any chemical reason that spectrum is affected by the adduct we are adding meaning that do we need to care about the adduct or not

Response: Even if you calculate neutral losses, **you still must care about what adduct was added**.

Neutral losses solve the simple arithmetic shift of parent fragmentations, but the adduct is not just an inert mass tag—it actively changes the underlying quantum mechanics and chemical pathways of how the molecule breaks apart.

Step 1: The Arithmetic vs. The Real Chemistry

- **Where Neutral Loss Helps (Pure Math):** If a molecule breaks at the exact same bond regardless of the adduct, the mass difference $\Delta m/z = m/z_{\text{precursor}} - m/z_{\text{fragment}}$ remains the same.
- **Where Neutral Loss Fails (Physical Chemistry):** The attached ion (H^+, Na^+, K^+, NH_4^+, etc.) physically alters the electron distribution, binding sites, and bond strengths across the entire molecule. Because the molecule now has different weak points, it breaks along completely different chemical pathways.

Step 2: Three Chemical Reasons the Spectrum Changes

1. Proton Mobility vs. Fixed Alkali Coordination (The "Mobile Proton" Model)

- **Protonated Molecules ($[M+H]^+$):** A proton (H^+) is tiny and highly mobile. In the collision chamber, thermal energy causes the proton to migrate across various heteroatoms (nitrogens, oxygens, carbonyls). Wherever the proton lands, it weakens adjacent covalent bonds, triggering diverse charge-directed cleavages throughout the whole skeleton.
- **Sodiated Molecules ($[M+Na]^+$):** Sodium (Na^+) is a much larger metal cation with fixed coordination preferences. It typically coordinates tightly between two or more electronegative oxygens (like a crown ether or chelation complex). It does not freely jump around. As a result, standard backbone cleavages are locked out, and you often see very little fragmentation, or entirely different ring cleavages that never occur in $[M+H]^+$.

2. Adduct Binding Affinity (Does the Adduct Stay or Fall Off?)

- **Sodium holds on tightly:** Covalent bonds in the molecule frequently break before the sodium ion falls off. The fragment you detect is $[\text{Fragment}+Na]^+$.
- **Ammonium falls off easily ($[M+NH_4]^+$):** The bond between ammonia and the molecule is relatively weak. In collision-induced dissociation, the adduct itself often detaches first as neutral ammonia:

  $$[M+NH_4]^+ \longrightarrow [M+H]^+ + NH_3 \quad (\text{Neutral loss of } 17.026 \text{ Da})$$

  From that point on, the fragmentation pattern resembles a protonated spectrum rather than an ammonium spectrum.

3. Charge Retention Asymmetry (Who Keeps the Charge?)

When a molecule breaks into two neutral pieces, A and B:

- Under $[M+H]^+$, piece A might have higher proton affinity, so $[A+H]^+$ is detected, while neutral B is invisible.
- Under $[M+Na]^+$, piece B might coordinate sodium much better, so $[B+Na]^+$ is detected instead. Because the mass spectrometer only detects charged fragments, the spectrum can look entirely inverted.

Step 3: A Concrete Numeric Example

Consider a simple ester molecule: $\text{R}_1\text{–COO–R}_2$.

Feature	Protonated Form ($[M+H]^+$)	Sodiated Form ($[M+Na]^+$)
Dominant Cleavage Mechanism	Proton lands on the ester oxygen; triggers facile cleavage into an acylium ion $[R_1\text{–CO}]^+$.	Na^+ chelates to both oxygens; the ester bond is stabilized against simple cleavage.
Observed Fragments	Strong peaks for $[R_1\text{–CO}]^+$ and loss of alcohol $R_2\text{OH}$.	Molecule resists fragmentation at low energy. At high energy, odd-electron cross-ring fractures or loss of CO_2 appear.
Neutral Loss Spectrum	Dominated by $\Delta m/z = \text{Mass}(R_2\text{OH})$.	Zero peak at $\Delta m/z = \text{Mass}(R_2\text{OH})$; dominated by entirely different $\Delta m/z$ values.

If you rely solely on matching neutral loss vectors without knowing the adduct, your model searches for an alcohol loss that does not exist in the sodiated spectrum.

Step 4: Practical Takeaway for Your Model

You cannot treat spectra across different adducts as interchangeable simply by subtracting precursor mass.

1. **Conditioning:** Supply the adduct type (or precursor ion ionization mode) as an explicit categorical input or embedding alongside the peak arrays.
2. **Database Filtering:** When querying COCONUT or PubChem by monoisotopic mass, you must know the adduct to calculate the correct neutral formula mass:

   $$\text{Neutral Mass} = \text{precursor_mz} - \text{adduct_mass}$$

   Subtracting 1.0078 (H^+) versus 22.9898 (Na^+) leads to completely different candidate pools.

User prompt: collision_energy_evCollision energy in electron volts, as a list. A single value ([20]) is one acquisition; several ([20, 40, 60, 80]) means the spectrum is merged from acquisitions at different energies. now how a spectrum with different collision_energy_ev is merged explain simply

Response: A merged spectrum is created by running the exact same molecule at multiple collision energies (e.g., gentle, medium, and harsh impacts) and combining all the resulting peaks into a single composite fingerprint.

Here is how that merging works step-by-step with simple numbers.

1. Why Laboratories Merge Spectra

- **At 20 eV (Low energy):** The molecule barely breaks. You get large fragments and the intact precursor, but you miss internal core details.
- **At 40 eV (Medium energy):** Functional groups and peripheral branches break off.
- **At 60 eV (High energy):** Tough rings and core structures shatter into tiny fragments, but larger pieces disappear completely.

Instead of storing 3 or 4 separate files, instruments or library curators merge them into **one unified "master spectrum"** that contains both the large diagnostic fragments and the small core fragments.

2. How the Merging Algorithm Works

Suppose an instrument runs the same compound at **20 eV** and **40 eV**:

- **Acquisition A (20 eV):**
  - Peak at $m/z = 105.07$ (Intensity = 0.20)
  - Peak at $m/z = 300.15$ (Intensity = 1.00) — *Large intact piece*
- **Acquisition B (40 eV):**
  - Peak at $m/z = 105.07$ (Intensity = 0.80) — *Broken down fragment*
  - Peak at $m/z = 77.04$ (Intensity = 0.40) — *Small ring fragment*

The merging pipeline follows three simple rules:

Step A: Grouping / Binning Close Masses

Because of slight detector calibration fluctuations, a peak measured at 105.0701 at 20 eV might register as 105.0703 at 40 eV. The algorithm groups peaks within an instrument tolerance window (e.g., ± 0.005 Da or ± 10 ppm) as the exact same fragment.

Step B: Combining Intensities

For peaks that appear in multiple acquisitions:

- If the peak is unique to one energy (like 77.04 or 300.15), it is kept as-is.
- If the peak appears in both runs (like 105.07), their intensities are typically **summed** or **averaged**:

$$\text{Raw Combined Intensity}(105.07) = 0.20 + 0.80 = 1.00$$

Step C: Final Re-normalization

After combining all peaks into a single list, the entire new list is divided by its maximum peak so the tallest peak resets to `1.0`:

Fragment m/z	Source	Merged Raw	Normalized Intensity
77.04	Seen only at 40 eV	0.40	0.40
105.07	Combined (20 eV + 40 eV)	1.00	1.00 (Base peak)
300.15	Seen only at 20 eV	1.00	1.00

The metadata is then recorded as `collision_energy_ev = [20, 40]`.

3. How to Handle This in Your Code

When feeding `collision_energy_ev` to a neural network, a Python list of variable length cannot be passed into a fixed linear layer directly. You can summarize the list into fixed scalar features:

```
def extract_energy_features(energy_list):
    if not energy_list or len(energy_list) == 0 or np.isnan(energy_list[0]):
        return [0.0, 0.0, 0.0, 1]  # [mean, min, max, is_missing_flag]

    energies = np.array(energy_list, dtype=float)
    return [
        float(energies.mean()),       # Average energy applied
        float(energies.min()),        # Lowest energy threshold
        float(energies.max()),        # Highest fragmentation force
        0                             # Present flag
    ]
```

This captures whether the spectrum is a single soft run ($[20]$) or a comprehensive merged acquisition ($[20, 40, 60, 80]$) without breaking batch tensor dimensions.
